Modelado Jerárquico

In [3]:
```
# ============================
# Entorno estable para PyMC + PreliZ en Colab
# ============================

!pip uninstall -y \
    preliz \
    pytensor \
    pytensor-distributions \
    arviz_stats \
    numba \
    llvmlite

!pip install -U --no-cache-dir \
    "numpy==2.0.2" \
    "scipy==1.16.1" \
    "pandas==2.3.0" \
    "matplotlib==3.10.0" \
    "llvmlite==0.43.0" \
    "numba==0.60.0" \
    "arviz==0.22.0" \
    "pymc==5.28.5" \
    "pytensor==2.38.3" \
    "arviz_stats==0.6.0" \
    "preliz==0.20.0"

print("\n================================")
print("IMPORTANTE")
print("Ahora hacé Runtime > Restart session")
print("y recién después ejecutá las celdas del notebook.")
print("================================")
```

```
Found existing installation: preliz 0.20.0
Uninstalling preliz-0.20.0:
  Successfully uninstalled preliz-0.20.0
Found existing installation: pytensor 2.38.3
Uninstalling pytensor-2.38.3:
  Successfully uninstalled pytensor-2.38.3
Found existing installation: arviz-stats 0.6.0
Uninstalling arviz-stats-0.6.0:
  Successfully uninstalled arviz-stats-0.6.0
Found existing installation: numba 0.60.0
Uninstalling numba-0.60.0:
  Successfully uninstalled numba-0.60.0
Found existing installation: llvmlite 0.43.0
Uninstalling llvmlite-0.43.0:
  Successfully uninstalled llvmlite-0.43.0
WARNING: Skipping pytensor-distributions as it is not installed.
```

```
Requirement already satisfied: numpy==2.0.2 in c:\trabajos\inferencia 2\venv\lib
\site-packages (2.0.2)
Requirement already satisfied: scipy==1.16.1 in c:\trabajos\inferencia 2\venv\lib
\site-packages (1.16.1)
Requirement already satisfied: pandas==2.3.0 in c:\trabajos\inferencia 2\venv\lib
\site-packages (2.3.0)
Requirement already satisfied: matplotlib==3.10.0 in c:\trabajos\inferencia 2\ven
v\lib\site-packages (3.10.0)
Collecting llvmlite==0.43.0
  Downloading llvmlite-0.43.0-cp311-cp311-win_amd64.whl (28.1 MB)
     ---------------------------------------- 28.1/28.1 MB 1.2 MB/s eta 0:00:00
Collecting numba==0.60.0
  Downloading numba-0.60.0-cp311-cp311-win_amd64.whl (2.7 MB)
     ---------------------------------------- 2.7/2.7 MB 1.5 MB/s eta 0:00:00
Requirement already satisfied: arviz==0.22.0 in c:\trabajos\inferencia 2\venv\lib
\site-packages (0.22.0)
Requirement already satisfied: pymc==5.28.5 in c:\trabajos\inferencia 2\venv\lib
\site-packages (5.28.5)
Collecting pytensor==2.38.3
  Downloading pytensor-2.38.3-cp311-cp311-win_amd64.whl (1.5 MB)
     ---------------------------------------- 1.5/1.5 MB 1.0 MB/s eta 0:00:00
Collecting arviz_stats==0.6.0
  Downloading arviz_stats-0.6.0-py3-none-any.whl (108 kB)
     ---------------------------------------- 108.8/108.8 kB 1.6 MB/s eta 0:00:00
Collecting preliz==0.20.0
  Downloading preliz-0.20.0-py3-none-any.whl (517 kB)
     ---------------------------------------- 517.6/517.6 kB 1.3 MB/s eta 0:00:00
Requirement already satisfied: python-dateutil>=2.8.2 in c:\trabajos\inferencia 2
\venv\lib\site-packages (from pandas==2.3.0) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\trabajos\inferencia 2\venv\lib
\site-packages (from pandas==2.3.0) (2026.3.post1)
Requirement already satisfied: tzdata>=2022.7 in c:\trabajos\inferencia 2\venv\li
b\site-packages (from pandas==2.3.0) (2026.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\trabajos\inferencia 2\venv
\lib\site-packages (from matplotlib==3.10.0) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\trabajos\inferencia 2\venv\lib
\site-packages (from matplotlib==3.10.0) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\trabajos\inferencia 2\venv
\lib\site-packages (from matplotlib==3.10.0) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\trabajos\inferencia 2\venv
\lib\site-packages (from matplotlib==3.10.0) (1.5.1)
Requirement already satisfied: packaging>=20.0 in c:\trabajos\inferencia 2\venv\l
ib\site-packages (from matplotlib==3.10.0) (26.3)
Requirement already satisfied: pillow>=8 in c:\trabajos\inferencia 2\venv\lib\sit
e-packages (from matplotlib==3.10.0) (12.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\trabajos\inferencia 2\venv
\lib\site-packages (from matplotlib==3.10.0) (3.3.2)
Requirement already satisfied: setuptools>=60.0.0 in c:\trabajos\inferencia 2\ven
v\lib\site-packages (from arviz==0.22.0) (65.5.0)
Requirement already satisfied: xarray>=2023.7.0 in c:\trabajos\inferencia 2\venv
\lib\site-packages (from arviz==0.22.0) (2026.7.0)
Requirement already satisfied: h5netcdf>=1.0.2 in c:\trabajos\inferencia 2\venv\l
ib\site-packages (from arviz==0.22.0) (1.8.1)
Requirement already satisfied: typing-extensions>=4.1.0 in c:\trabajos\inferencia
2\venv\lib\site-packages (from arviz==0.22.0) (4.16.0)
Requirement already satisfied: xarray-einstats>=0.3 in c:\trabajos\inferencia 2\v
env\lib\site-packages (from arviz==0.22.0) (0.9.1)
Requirement already satisfied: cachetools<7,>=4.2.1 in c:\trabajos\inferencia 2\v
env\lib\site-packages (from pymc==5.28.5) (6.2.6)
Requirement already satisfied: cloudpickle in c:\trabajos\inferencia 2\venv\lib\s
```

```
ite-packages (from pymc==5.28.5) (3.1.2)
Requirement already satisfied: rich>=13.7.1 in c:\trabajos\inferencia 2\venv\lib
\site-packages (from pymc==5.28.5) (15.0.0)
Requirement already satisfied: threadpoolctl<4.0.0,>=3.1.0 in c:\trabajos\inferen
cia 2\venv\lib\site-packages (from pymc==5.28.5) (3.6.0)
Requirement already satisfied: filelock>=3.15 in c:\trabajos\inferencia 2\venv\li
b\site-packages (from pytensor==2.38.3) (3.32.4)
Requirement already satisfied: etuples in c:\trabajos\inferencia 2\venv\lib\site-
packages (from pytensor==2.38.3) (0.3.10)
Requirement already satisfied: logical-unification in c:\trabajos\inferencia 2\ve
nv\lib\site-packages (from pytensor==2.38.3) (0.4.7)
Requirement already satisfied: miniKanren in c:\trabajos\inferencia 2\venv\lib\si
te-packages (from pytensor==2.38.3) (1.0.5)
Requirement already satisfied: cons in c:\trabajos\inferencia 2\venv\lib\site-pac
kages (from pytensor==2.38.3) (0.4.7)
Requirement already satisfied: six>=1.5 in c:\trabajos\inferencia 2\venv\lib\site
-packages (from python-dateutil>=2.8.2->pandas==2.3.0) (1.17.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\trabajos\inferencia 2
\venv\lib\site-packages (from rich>=13.7.1->pymc==5.28.5) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\trabajos\inferencia
2\venv\lib\site-packages (from rich>=13.7.1->pymc==5.28.5) (2.21.0)
Requirement already satisfied: toolz in c:\trabajos\inferencia 2\venv\lib\site-pa
ckages (from logical-unification->pytensor==2.38.3) (1.1.0)
Requirement already satisfied: multipledispatch in c:\trabajos\inferencia 2\venv
\lib\site-packages (from logical-unification->pytensor==2.38.3) (1.0.0)
Requirement already satisfied: mdurl~=0.1 in c:\trabajos\inferencia 2\venv\lib\si
te-packages (from markdown-it-py>=2.2.0->rich>=13.7.1->pymc==5.28.5) (0.1.2)
Installing collected packages: llvmlite, numba, arviz_stats, preliz, pytensor
Successfully installed arviz_stats-0.6.0 llvmlite-0.43.0 numba-0.60.0 preliz-0.2
0.0 pytensor-2.38.3

==============================
IMPORTANTE
Ahora hacé Runtime > Restart session
y recién después ejecutá las celdas del notebook.
==============================
[notice] A new release of pip available: 22.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [1]:
```python
import arviz as az
import pymc as pm
import preliz as pz

import numpy as np
import scipy.stats as stats
import pandas as pd
import matplotlib.pyplot as plt

az.style.use("arviz-doc")
```

```
WARNING (pytensor.configdefaults): g++ not available, if using conda: `conda inst
all gxx`
WARNING (pytensor.configdefaults): g++ not detected!  PyTensor will be unable to
compile C-implementations and will default to Python. Performance may be severely
degraded. To remove this warning, set PyTensor flags cxx to an empty string.
c:\Trabajos\Inferencia 2\venv\Lib\site-packages\preliz\ppls\pymc_io.py:12: Future
Warning: `pytensor.graph.basic.ancestors` was moved to `pytensor.graph.traversal.
ancestors`. Calling it from the old location will fail in a future release.
  from pytensor.graph.basic import ancestors
```

1) El Modelo Jerárquico:

En la mayoría de los casos de estudio, los datos con los que trabajamos podrían presentar un agrupamiento o jerarquía incluso muchas veces dada por distintos niveles. En este capítulo estudiaremos dos herramientas que nos serán útiles para abordar este tipo de casos, que son el ***modelado jerárquico*** (con agrupamiento total y parcial) y el posible ***efecto de contracción***.

En estos casos, para trabajar con la distribución a posteriori se introducen los llamados ***hiper-priors*** para poder trabajar a partir de ellos con estimaciones por cada grupo. Esto es, inducimos una agrupación parcial en un punto entre el modelo agrupado y el modelo no agrupado.

Consideremos el estudio del desempeño de alumnos de diferentes escuelas a nivel provincial. Podŕiamos tener información dada por escuelas de distintas ciudades. En este caso tenemos un agrupamiento o jerarquía dado "naturalmente" en diferentes niveles: *están agrupados por escuela, ciudad y provincia.*

¿Cómo sería el caso de un estudio de salud teniendo en cuenta la información en hospitales?

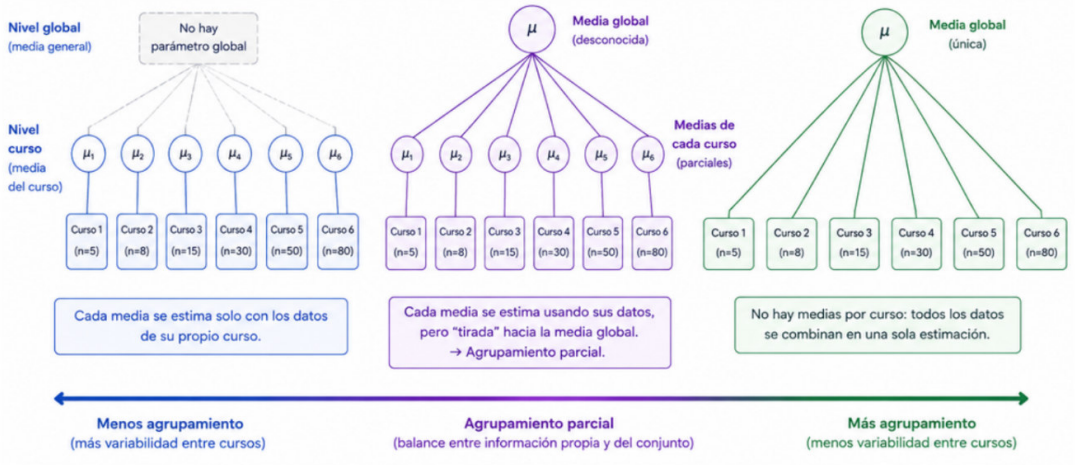
En muchos casos, tener en cuenta esta situación de agrupamiento no es tan fácil de ver, y muchas veces se ignora por completo, pero no es menor que exista esta estructura jerárquica porque analizar los datos por separado o como un todo, no siempre refleja la relación real de los mismos, ni nos permite obtener estimaciones tan precisas.

El costo de no trabajar con Modelos jerárquicos cuando es debido, es perder detalles y características propias de cada grupo, e incluso diferencias entre ellos, es decir, *se eliminaría la particularidad de la naturaleza de los datos formados por grupos diferentes, pero que al mismo tiempo comparten información.*

NOTA: los Modelos Jerárquicos también son conocidos en la literatura cn los siguientes nombres: *modelos multinivel, modelos de efectos mixtos, modelos de efectos aleatorios o modelos anidados.*

¿Cómo se construye un modelo jerárquico?

La idea central de la construcción de un modelo jerárquico, es **asignar distribuciones a priori a las distribuciones a priori** , lo que se engloba en la utilización de los hiper-priors, con los que estamos asumiendo que los priors de cada grupo no son nesariamente iguales pero sí se relacionan ya que provienen de una población común.



El problema de los renacuajos

Estudiemos la supervivencia de renacuajos en 48 tanques.

En cada tanque se colocó una cantidad conocida de renacuajos y se registró cuántos sobrevivieron.

Queremos modelar la probabilidad de supervivencia de los renacuajos teniendo en cuenta las condiciones ambientales como la temperatura, el pH, la luz, etc. y cómo las mismas varían entre cada uno de los tanques.

¿Por qué necesitamos un modelo jerárquico?:

porque las condiciones anteriores nos permiten pensar en cada tanque como un grupo, siendo la población común de la que provienen los renacuajos.

NOTA: pueden estudiar este ejemplo completo de statistical rethinking

In [6]:
```
# from google.colab import drive
# drive.mount('/content/drive')
```

En primer lugar, leeremos los datos y estudiaremos un poco lo que contiene la tabla:

In [7]:
```
d = pd.read_csv('renacuajos.csv', sep=",")
coords = {"tanks": list(d.index)}
```

In [8]:
```
d.head()
```

Out[8]:

	density	pred	size	surv	propsurv
0	10	no	big	9	0.9
1	10	no	big	10	1.0
2	10	no	big	7	0.7
3	10	no	big	10	1.0
4	10	no	small	9	0.9

Tenemos las siguientes variables:

- `tank:` identificador del tanque. Indica a qué tanque pertenece cada observación (hay 48 tanques).
- `density:` cantidad de renacuajos colocados inicialmente en el tanque, es decir, la densidad inicial.
- `surv:` cantidad de renacuajos que sobrevivieron al finalizar el experimento.
- `propsurv:` proporción de renacuajos que sobrevivieron en cada tanque. Se calcula como:

$$propsurv = \frac{surv}{density}$$

Construímos el modelo:

Suponemos que, para cada tanque $i = 1, \ldots, 48,$ el número de renacuajos sobrevivientes sigue una distribución Binomial

$$surv_i \sim Binomial(n_i, p_i)$$

donde:

- $surv_i$ representa la cantidad de renacuajos que sobrevivieron en el tanque i;
- n_i es la cantidad inicial de renacuajos en dicho tanque (variable density);
- p_i es la probabilidad de supervivencia correspondiente al tanque i.

Este modelo es adecuado porque conocemos cuántos renacuajos hay y queremos estimar la probabilidad de supervivencia.

Hasta acá, como se puede ver, es simplemente un modelo Binomial para cada uno de los tanques.

¿Cuál es el problema de dejar el modelo tal cual está definido?:

Pensemos en que, si estimamos una probabilidad p_i diferente para cada tanque, estamos asumiendo que todos los tanques son completamente independientes, pero en realidad la intuición nos hace comenzar a plantearnos que:

1. Algunos tanques tienen muy pocos individuos, otros muchos.
2. Las estimaciones pueden ser muy inestables.

Entonces necesitamos compartir información.

De este modo nuestro modelos "nos queda un poco débil", y es así como aparece la necesidad del modelo jerárquico para poder incluir las dos situaciones anteriores.

Para cada tanque $i = 1, \ldots, 48,$, al igual que en el caso previo, el número de renacuajos sobrevivientes sigue una distribución Binomial dada por

$$surv_i \sim Binomial(n_i, p_i)$$

- $surv_i$ representa la cantidad de renacuajos que sobrevivieron en el tanque i;
- n_i es la cantidad inicial de renacuajos en dicho tanque (variable density);
- p_i es la probabilidad de supervivencia correspondiente al tanque i.

Como una probabilidad debe tomar valores entre 0 y 1, modelamos p_i mediante una transformación logística aplicada a un parámetro sin restricciones:

$$p_i = logit^{-1}(\alpha_i) = \frac{1}{1 + e^{\alpha_i}}$$

Los efectos de cada tanque se consideran provenientes de una distribución común,

$$\alpha_i \sim N(\mu, \sigma)$$

donde:

- μ representa el efecto promedio de todos los tanques;
- σ cuantifica la variabilidad entre tanques.

Finalmente, se asignan distribuciones a priori a los hiperparámetros,

$$\mu \sim N(0, 2)$$

$$\sigma \sim HalfNormal(2)$$

Resumen del modelo:

$$surv_i \sim Binomial(n_i, p_i)$$

$$p_i = logit^{-1}(\alpha_i) = \frac{1}{1 + e^{\alpha_i}}$$

$$\alpha_i \sim N(\mu, \sigma)$$

$$\mu \sim N(0, 2)$$

$$\sigma \sim HalfNormal(2)$$

$i = 1, \ldots, 48.$

Pensemos cómo se componen los niveles de este modelo:

Podemos pensarlo en tres niveles principales:

1. *Nivel de los datos:* para cada tanque observamos cuántos renacuajos sobrevivieron de los que fueron colocados inicialmente. Por eso usamos una distribución Binomial:

$$surv_i \sim Binomial(n_i, p_i)$$

2. *Nivel de los tanques:* cada tanque tiene su propia probabilidad de supervivencia p_i, que se expresa mediante α_i. Sin embargo, estos efectos no se estiman de manera completamente independiente, sino que provienen de una distribución común:

$$\alpha_i \sim N(\mu, \sigma)$$

   Así, los tanques comparten información.

3. *Nivel de la población de tanques:* los parámetros que describen esa distribución común, μ y σ, también tienen sus propias distribuciones a priori:

$$\mu \sim N(0, 2)$$

$$\sigma \sim HalfNormal(2)$$

NOTA: tenemos observaciones dentro de tanques, tanques dentro de una población común. Esta estructura permite que los tanques con pocos datos puedan beneficiarse de la información de los demás, evitando estimaciones demasiado extremas o inestables.

Intercambiabilidad:

Hasta este momento hemos considerado que los efectos asociados a los distintos tanques pueden describirse mediante una misma distribución de probabilidad. Esta suposición recibe el nombre de **intercambiabilidad**.

De manera formal, una colección de variables aleatorias $(X_1, X_2, \ldots, X_J)$, ya sea finita o infinita, es **intercambiable** cuando su distribución conjunta no cambia al modificar el orden de las variables. Es decir, podemos permutar sus índices sin alterar el modelo probabilístico.

Llevando esta idea a nuestro ejemplo, estamos suponiendo que, antes de observar los resultados, no contamos con información que nos permita considerar a un tanque como sistemáticamente diferente de los demás.

Por ejemplo, si quitáramos las etiquetas que identifican a cada tanque y solo conserváramos la cantidad de semillas que sobrevivieron en cada uno, no tendríamos motivos para pensar que el tanque 3 tiene una probabilidad de supervivencia diferente de la del tanque 18 simplemente por ocupar una determinada posición o tener un determinado número.

Para verlo con mayor claridad, supongamos que realizamos el experimento utilizando diez tanques y que el resultado correspondiente al tanque 4 permanece oculto. Para los otros nueve tanques conocemos las proporciones de supervivencia:

Tanque	Proporción
1	0.62
2	0.55
3	0.58
5	0.61
6	0.59
7	0.53
8	0.57
9	0.60
10	0.56

¿Qué podemos decir entonces sobre el tanque 4?

Bajo la hipótesis de **intercambiabilidad**, no podemos conocer exactamente su proporción de supervivencia, pero sí podemos utilizar la información de los otros tanques para obtener una expectativa razonable sobre su resultado.

Ahora bien, si supiéramos que el tanque 4 estuvo sometido a condiciones diferentes, ya no sería razonable tratarlo como intercambiable con los demás. Esa información debería incorporarse al modelo.

Por último, **intercambiabilidad no significa independencia**. Una secuencia iid siempre es intercambiable, pero una secuencia intercambiable no necesariamente es independiente. Esta distinción es fundamental en los **modelos jerárquicos**, porque permite compartir información entre los distintos grupos.

Teorema de De Finetti:

Veamos un teorema muy importante que fundamenta matemáticamente la hipótesis de intercambiabilidad:

Este resultado se conoce como ***Teorema de De Finetti*** y establece que una secuencia de variables intercambiables se puede representar como variables condicionalmente independientes dado un parámetro latente.

Esto es:

una secuencia de variables aleatorias es intercambiable si y solo si para todo n podemos escribir:

$$p(X_1, X_2, \ldots, X_n) = \int \prod_i^n p(X_i \mid \theta)\, p(\theta)$$

Es decir, que un conjunto de variable aleatorias puede ser descrito por:

- un parámetro θ
- un likelihood $p(X \mid \theta)$
- un prior $p(\theta)$

Supongamos ahora que θ representa un conjunto de parámetros, $\theta = (\theta_1, \theta_2, \ldots, \theta_n)$ y que este conjunto de parámetros es intercambiable. Entonces podemos escribir:

$$(\theta_1, \theta_2, \ldots, \theta_n) = \int \prod_i^n p(\theta_i \mid \psi)\, p(\psi)$$

Obteniendo un modelo jerárquico que podríamos reescribir como:

$$X_{ij} \sim p(X \mid \theta_i)$$
$$\theta_i \sim p(\theta \mid \psi)$$
$$\psi \sim p(\psi)$$

```
       ψ ~ p(ψ)                Nivel 3: Hiperpriors (μ, σ de la
población)
          |
          ▼
     θ_i ~ p(θ | ψ)            Nivel 2: Parámetro de cada grupo (α_i de
cada invernadero)
          |
          ▼
   X_ij ~ p(X | θ_i)           Nivel 1: Datos observados (semilla j en
invernadero i)

          ψ = (μ, σ) ~ p(ψ)             Nivel 3: Hiperpriors (μ ~
Normal, σ ~ HalfNormal)
                    |
                    ▼
           α_i ~ Normal(μ, σ)           Nivel 2: Parámetro del tanque i
(escala logit)
                    |
                    ▼
       surv_i ~ Binomial(n_i, p_i)      Nivel 1: Datos observados
(renacuajos supervivientes surv_i de n_i)
```

In [9]:
```python
with pm.Model(coords=coords) as modelo_renacuajos:
    # Hiperpriors
    μ = pm.Normal('μ', 0., 2.)
    σ = pm.HalfNormal('σ', 2.)
```

```
    # Prior
    α_tanque = pm.Normal('α_tanque', μ, σ, dims="tanks")
    p = pm.Deterministic('p', pm.math.sigmoid(α_tanque))  # transformación logís
    #likelihood
    surv = pm.Binomial('surv', n=d.density, p=p, observed=d.surv)

    idata_renacuajos = pm.sample(2000, tune=2000)
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [μ, σ, α_tanque]
```

Output()

```
Sampling 4 chains for 2_000 tune and 2_000 draw iterations (8_000 + 8_000 draws t
otal) took 209 seconds.
```

In [10]:

```
# pm.model_to_graphviz(modelo_renacuajos)


import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, Rectangle
import pymc as pm

def plot_modelo_jerarquico(model, figsize=(5.2, 4.2), dpi=120):
    """
    Dibuja el grafo jerárquico con los nombres y fórmulas reales del modelo PyMC
    """
    # 1. Obtener dimensión de grupos
    n_groups = "N"
    if "tanks" in model.coords:
        n_groups = str(len(model.coords["tanks"]))
    elif len(model.observed_RVs) > 0:
        n_groups = str(len(model.observed_RVs[0].eval()))

    # 2. Extraer nombres y distribuciones del modelo
    # Hiperparámetros
    mu_name = model.free_RVs[0].name if len(model.free_RVs) > 0 else "μ"
    sigma_name = model.free_RVs[1].name if len(model.free_RVs) > 1 else "σ"

    # Efectos de grupo
    alpha_name = model.free_RVs[2].name if len(model.free_RVs) > 2 else "α_tanqu

    # Determinística (probabilidad)
    p_name = model.deterministics[0].name if len(model.deterministics) > 0 else

    # Observada
    obs_name = model.observed_RVs[0].name if len(model.observed_RVs) > 0 else "s

    # Etiquetas completas con fórmulas
    labels = {
        "mu": f"{mu_name} ~ Normal(0, 2)",
        "sigma": f"{sigma_name} ~ HalfNormal(2)",
        "alpha": f"{alpha_name} ~ Normal({mu_name}, {sigma_name})",
        "p": f"{p_name} = sigmoid({alpha_name})",
        "surv": f"{obs_name} ~ Binomial(n, {p_name})"
    }

    fig, ax = plt.subplots(figsize=figsize, dpi=dpi)

    # Posiciones de los nodos
```

```
    nodes = {
        "mu": (-1.1, 3.0, labels["mu"], "latent"),
        "sigma": (1.1, 3.0, labels["sigma"], "latent"),
        "alpha": (0.0, 2.0, labels["alpha"], "latent"),
        "p": (0.0, 1.0, labels["p"], "det"),
        "surv": (0.0, 0.0, labels["surv"], "obs"),
    }

    # Dibujar nodos con texto
    for k, (x, y, text, n_type) in nodes.items():
        fc = "#fee6ce" if n_type == "obs" else "#deebf7" if n_type == "det" else
        ax.text(x, y, text, ha="center", va="center", fontsize=8.5,
                bbox=dict(boxstyle="round,pad=0.35", fc=fc, ec="black", lw=1))

    # Flechas
    arrows = [
        ((-0.7, 2.75), (-0.2, 2.25)),
        ((0.7, 2.75), (0.2, 2.25)),
        ((0.0, 1.75), (0.0, 1.25)),
        ((0.0, 0.75), (0.0, 0.25)),
    ]

    for start, end in arrows:
        ax.add_patch(FancyArrowPatch(start, end, arrowstyle="->", mutation_scale

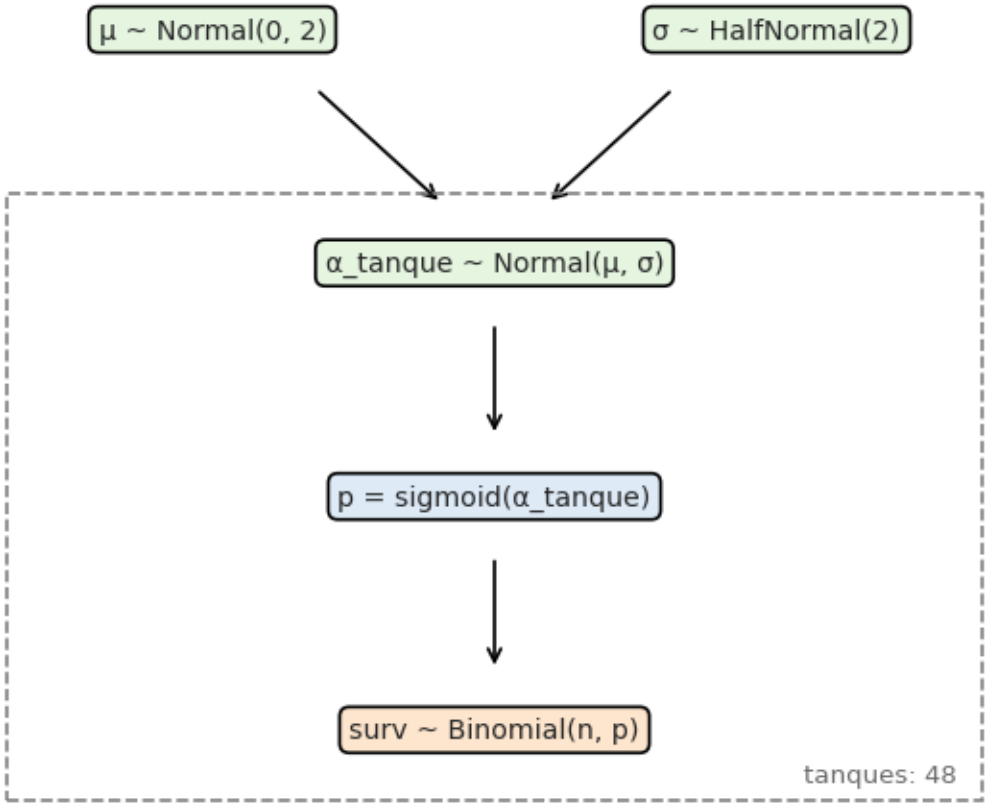
    # Placa con etiqueta de dimensión
    ax.add_patch(Rectangle((-1.9, -0.3), 3.8, 2.6, fill=False, ls="--", color="g
    ax.text(1.8, -0.22, f"tanques: {n_groups}", fontsize=8, color="dimgray", ha=

    ax.set_xlim(-2.2, 2.2)
    ax.set_ylim(-0.45, 3.4)
    ax.axis("off")
    plt.tight_layout()
    return fig, ax
```

In [11]:
```
plot_modelo_jerarquico(modelo_renacuajos)
```

```
C:\Users\MAURICIO\AppData\Local\Temp\ipykernel_5104\444511489.py:77: UserWarning:
The figure layout has changed to tight
  plt.tight_layout()
```

Out[11]:
```
(<Figure size 624x504 with 1 Axes>, <Axes: >)
```



En la siguiente figura se comparan las **proporciones observadas** de supervivencia en cada tanque (puntos azules) con las **estimadas por el modelo** (puntos turquesa).

La línea punteada representa la proporción promedio de supervivencia considerando todos los tanques, mientras que las líneas verticales separan los tanques según su densidad inicial: 10, 25 y 35 renacuajos.

Observamos que las estimaciones del modelo se encuentran, en general, más cerca del promedio global que las proporciones observadas. Esto es consecuencia de la **información compartida entre los tanques** y se conoce como **contracción hacia la media**.

definimo suna función logística:

In [12]:
```
def logistic(x):
    return 1 / (1 + np.exp(-x))
```

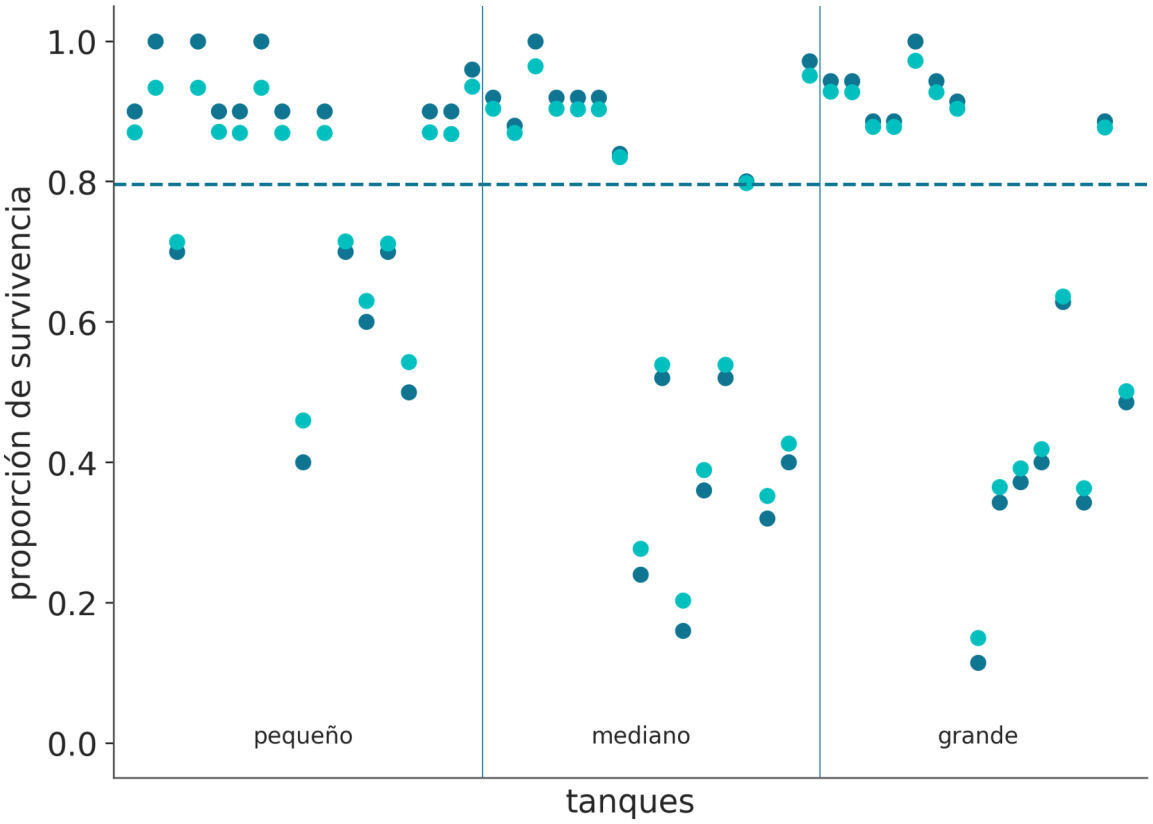
In [13]:
```
_, ax = plt.subplots(1, 1, figsize=(7, 5))

post_r = az.extract(idata_renacuajos)

ax.scatter(np.arange(0, 48), d.propsurv, color='C0')
ax.scatter(np.arange(0, 48), post_r['p'].mean("sample"), color='C1')
ax.hlines(logistic(post_r['μ'].median("sample")), -1, 49, linestyles='--')

idx = d.density[d.density.diff() > 0].index
ax.vlines(idx + 0.5, -.05, 1.05, lw=.5)
for i, t in zip(np.linspace(0, 48, 7)[1::2], ('pequeño', 'mediano', 'grande')):
```

```
    ax.text(i, 0, t, horizontalalignment='center')
ax.set_xlabel('tanques')
ax.set_ylabel('proporción de survivencia')
ax.set_xlim(-1, 48)
ax.set_xticks([])
ax.set_ylim(-.05, 1.05)
ax.grid(False)
```



Caso 1. Modelo no jerárquico (sin pooling)

Veamos un primer modelo, pero con cada tanque teniendo su propia probabilidad independiente, es decir, cada tanque tiene su propia probabilidad de supervivencia y no comparte información con ningún otro tanque.

In [14]:
```
coords = {"tanks": list(d.index)}

with pm.Model(coords=coords) as modelo_no_jerarquico:

    # Un parámetro independiente para cada tanque
    p = pm.Beta("p", alpha=1, beta=1, dims="tanks")

    # Likelihood
    surv = pm.Binomial(
        "surv",
        n=d.density,
        p=p,
        observed=d.surv
    )

    idata_no_jerarquico = pm.sample(
        2000,
        tune=2000,
```

```
        random_seed=123
    )
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [p]
```

Output()

```
Sampling 4 chains for 2_000 tune and 2_000 draw iterations (8_000 + 8_000 draws t
otal) took 189 seconds.
```

Caso 2. Modelo jerárquico (con pooling parcial)

In [15]:
```
with pm.Model(coords=coords) as modelo_jerarquico:

    μ = pm.Normal("μ", 0, 2)
    σ = pm.HalfNormal("σ", 2)

    α_tanque = pm.Normal(
        "α_tanque",
        μ,
        σ,
        dims="tanks"
    )

    p = pm.Deterministic(
        "p",
        pm.math.sigmoid(α_tanque)
    )

    surv = pm.Binomial(
        "surv",
        n=d.density,
        p=p,
        observed=d.surv
    )

    idata_jerarquico = pm.sample(
        2000,
        tune=2000,
        random_seed=123
    )
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [μ, σ, α_tanque]
```

Output()

```
Sampling 4 chains for 2_000 tune and 2_000 draw iterations (8_000 + 8_000 draws t
otal) took 244 seconds.
```

Comparemos ambos modelos:

In [16]:
```
post_no = az.extract(idata_no_jerarquico)
post_h = az.extract(idata_jerarquico)

fig, ax = plt.subplots(figsize=(7,5))

ax.scatter(
```

```
    np.arange(48),
    d.propsurv,
    color="C0",
    label="Observado"
)

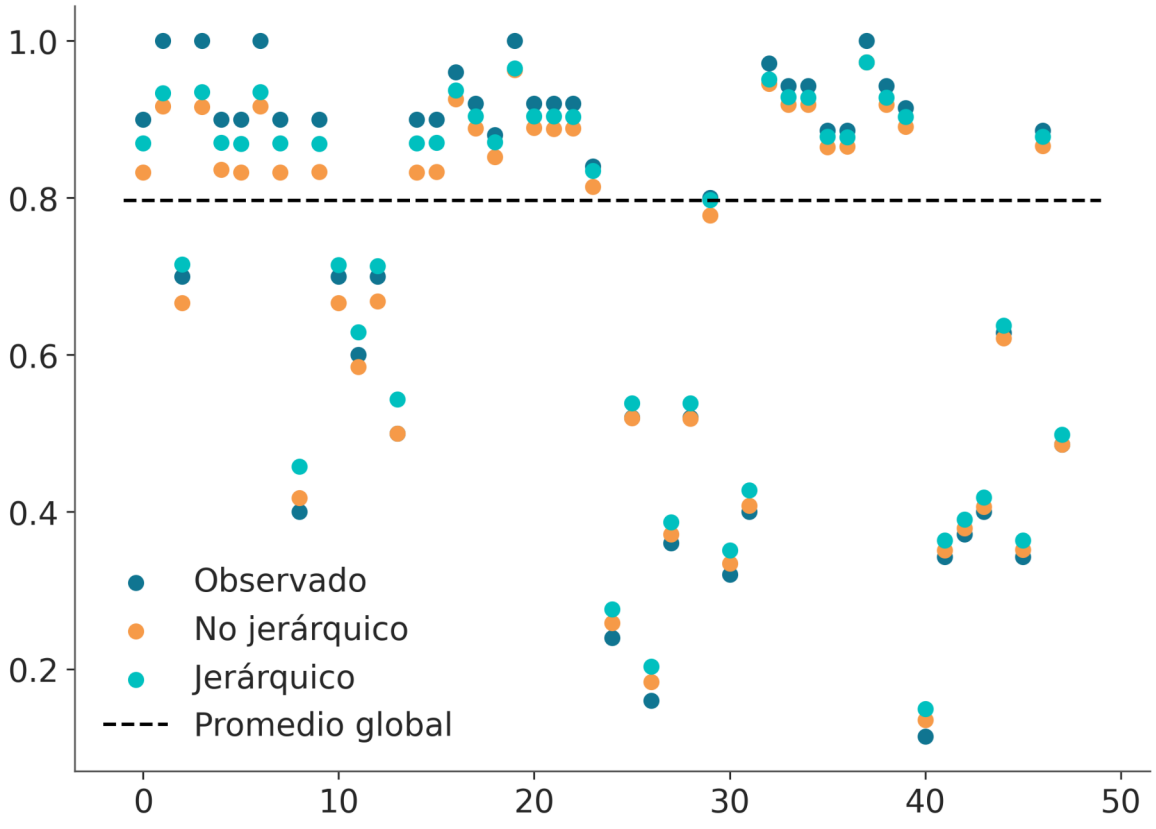
ax.scatter(
    np.arange(48),
    post_no["p"].mean("sample"),
    color="C2",
    label="No jerárquico"
)

ax.scatter(
    np.arange(48),
    post_h["p"].mean("sample"),
    color="C1",
    label="Jerárquico"
)

ax.hlines(
    logistic(post_h["μ"].median("sample")),
    -1,
    49,
    ls="--",
    color="k",
    label="Promedio global"
)

ax.legend()
```

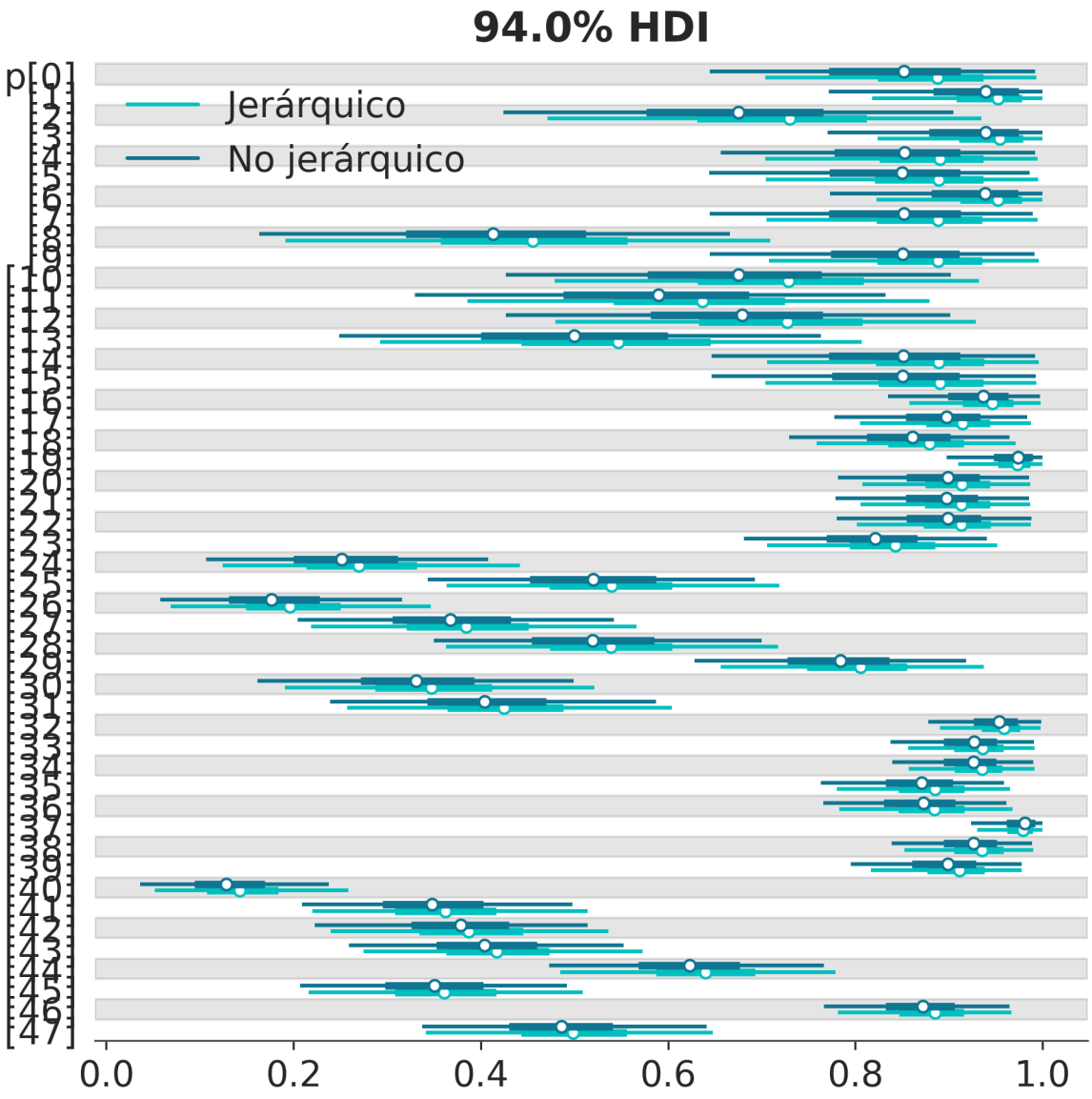
Out[16]: <matplotlib.legend.Legend at 0x2861c54ea50>



In [17]:
```
az.plot_forest(
    [idata_no_jerarquico,
```

```
    idata_jerarquico],
    model_names=[
        "No jerárquico",
        "Jerárquico"
    ],
    var_names="p",
    combined=True,
    figsize=(6,6)
)
```

Out[17]: array([<Axes: title={'center': '94.0% HDI'}>], dtype=object)



Observar que este último gráfico muestra muy bien cómo las estimaciones cambian cuando los grupos comparten información.

El gráfico permite comparar un modelo **sin pooling** con uno de **pooling parcial**.

1. En el modelo no jerárquico, cada tanque se estima de manera independiente, por lo que los tanques con pocos datos presentan mayor variabilidad.

2. El modelo jerárquico permite **compartir información entre los tanques**. Así, las estimaciones con poca evidencia se acercan al promedio global, mientras que las que cuentan con más datos permanecen más próximas a lo observado.

¡¡Este fenómeno se conoce como ***contracción o shrinkage***!!

Enfoque	Hacia dónde es atraída la estimación	¿Por qué?
No Jerárquico (Naranja)	Hacia $\mathbf{0.50}$	Cada tanque está aislado con su prior plano $\text{Beta}(1, 1)$.
Jerárquico (Turquesa)	Hacia $\mathbf{0.80}$ (Media global)	Los tanques comparten información y aprenden el promedio de toda la población.

Ejemplo para resolver en clase:

Una empresa agrícola desea evaluar la calidad de un nuevo tratamiento para semillas. Para ello, se sembraron semillas tratadas en distintos invernaderos. En cada uno se registró si cada semilla logró germinar correctamente.

Cada observación se codifica como:

- 1: la semilla germinó correctamente.
- 0: la semilla no germinó.

Los datos se encuentran agrupados por invernadero. Aunque cada invernadero puede presentar condiciones particulares de temperatura, humedad o iluminación, se supone que todos pertenecen a una misma población y, por lo tanto, resulta razonable modelarlos mediante un enfoque jerárquico.

Para este ejercicio se utilizarán datos sintéticos correspondientes a cuatro invernaderos.

In [2]:
```
# Cantidad de semillas evaluadas por invernadero
N = [20, 30, 25, 40]

# Cantidad de semillas germinadas
G = [15, 18, 11, 34]

group_idx = np.repeat(np.arange(len(N)), N)

data = []

for i in range(len(N)):
    data.extend(np.repeat([1, 0], [G[i], N[i]-G[i]]))

data = np.array(data)
```

Actividades:

a) Plantee un modelo jerárquico bayesiano para estimar la probabilidad de germinación de cada invernadero. Describa el significado de cada uno de los parámetros e indique qué distribución asignaría a cada nivel del modelo.

b) Implemente el modelo utilizando PyMC y obtenga muestras de la distribución posterior.

c) Obtenga un resumen posterior de las probabilidades de germinación de cada invernadero y de los hiperparámetros del modelo. ¿Qué información aporta cada uno de ellos?

d) Repita el análisis modificando únicamente los datos del último invernadero para obtener los siguientes escenarios

shape recibe un número entero (o una tupla de enteros) para definir directamente cuántos elementos tiene la variable (por ejemplo, shape=4).

dims recibe el nombre de una dimensión en forma de texto (string), la cual previamente tenés que haber registrado en las coordenadas del modelo (coords={"invernaderos": [...]}).

In [3]:
```
# Escenario A
G = [15, 18, 11, 34]
#invernaderos = [1,2,3,4]

with pm.Model() as model_a:
    # hiperpriors - priors de priors
    mu = pm.Normal("mu", mu=0, sigma=2)
    sigma = pm.HalfNormal("sigma", sigma=2)

    # prior
    p = pm.Normal("p", mu=mu, sigma=sigma, shape = len(N))
    p_normalizado = pm.Deterministic("p_normalizado", pm.math.sigmoid(p))
    '''markdown
    Se toma el espacio sin restricciones ($\alpha_i$): La distribución Normal ($
    vive en la recta real (va de $-\infty$ a $+\infty$).
    Es perfecta para aplicar jerarquía, promediar, sumar y calcular desviaciones
    Usás la función sigmoidea (o logit inverso) para transformar ese valor norma
    '''

    #likelihood
    likelihood = pm.Binomial("likelihood", n=N, p=p_normalizado, observed=G)
```

In [4]:
```
with model_a:
    idata = pm.sample(draws = 2000, tune = 2000, random_seed=123)
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [mu, sigma, p]
```

Output()

```
Sampling 4 chains for 2_000 tune and 2_000 draw iterations (8_000 + 8_000 draws t
otal) took 40 seconds.
There were 197 divergences after tuning. Increase `target_accept` or reparameteri
ze.
```

In [21]:
```
#modo bernoulli
# 2. Transformar a formato individual (Bernoulli)
# group_idx = np.repeat(np.arange(len(N)), N)

# data = []
# for i in range(len(N)):
#     data.extend(np.repeat([1, 0], [G[i], N[i]-G[i]]))
# data = np.array(data)
```

```
# # 3. Modelo en PyMC
# with pm.Model() as model_bernoulli:
#     # Hiperpriors
#     mu = pm.Normal("mu", mu=0, sigma=2)
#     sigma = pm.HalfNormal("sigma", sigma=2)

#     # Prior por invernadero (usando shape en vez de dims)
#     p = pm.Normal("p", mu=mu, sigma=sigma, shape=len(N))
#     p_normalizado = pm.Deterministic("p_normalizado", pm.math.sigmoid(p))

#     # Likelihood con Bernoulli
#     # Usamos p_normalizado[group_idx] para asignarle la probabilidad 'p'
#     # correspondiente a cada semilla según su invernadero
#     likelihood = pm.Bernoulli("likelihood", p=p_normalizado[group_idx], observ

#     # Muestreo
#     idata = pm.sample(draws=2000, tune=2000, random_seed=123)
```

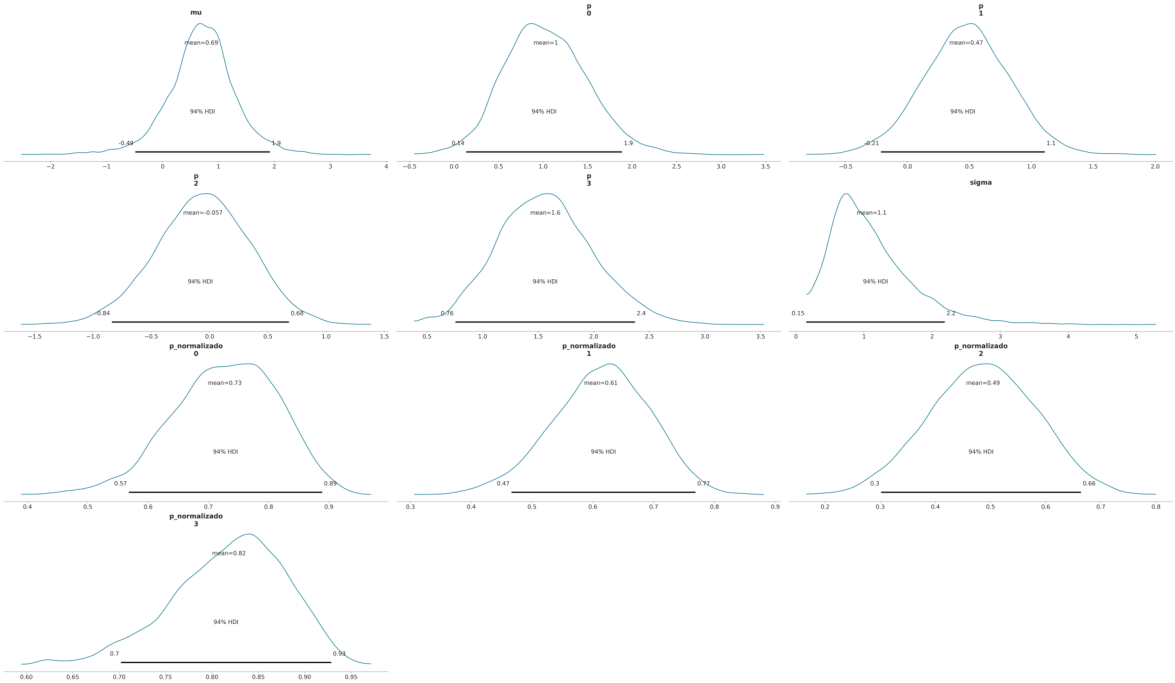
In [5]:
```
# Obtener el resumen de la posterior para el modelo A
az.summary(idata, var_names=["mu", "sigma", "p_normalizado"])
```

Out[5]:

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_t
mu	0.692	0.621	-0.488	1.919	0.009	0.011	4955.0	414
sigma	1.111	0.624	0.153	2.185	0.015	0.009	705.0	23
p_normalizado[0]	0.727	0.088	0.568	0.889	0.001	0.001	6146.0	570
p_normalizado[1]	0.613	0.081	0.466	0.769	0.001	0.001	6360.0	504
p_normalizado[2]	0.486	0.097	0.302	0.664	0.002	0.001	2854.0	253
p_normalizado[3]	0.818	0.062	0.702	0.928	0.002	0.002	1214.0	51

In [6]:
```
az.plot_posterior(idata)
```

Out[6]:
```
array([[<Axes: title={'center': 'mu'}>, <Axes: title={'center': 'p\n0'}>,
        <Axes: title={'center': 'p\n1'}>],
       [<Axes: title={'center': 'p\n2'}>,
        <Axes: title={'center': 'p\n3'}>,
        <Axes: title={'center': 'sigma'}>],
       [<Axes: title={'center': 'p_normalizado\n0'}>,
        <Axes: title={'center': 'p_normalizado\n1'}>,
        <Axes: title={'center': 'p_normalizado\n2'}>],
       [<Axes: title={'center': 'p_normalizado\n3'}>, <Axes: >, <Axes: >]],
      dtype=object)
```



In [23]:
```
# Escenario B
G = [15, 18, 11, 8]
```

In [24]:
```
# Escenario C
G = [15, 18, 11, 20]
```

Compare las estimaciones obtenidas para los tres escenarios y explique cómo influye la información proveniente de los demás invernaderos sobre la estimación del último grupo.

e) A partir de los resultados obtenidos, responda:

¿En qué situaciones el modelo jerárquico produce una mayor contracción (shrinkage)?

¿Qué ventajas presenta este enfoque frente a estimar cada invernadero de manera independiente?

¿En qué circunstancias dejaría de ser razonable asumir que los invernaderos son intercambiables?

Resolución:

Tenemos cuatro invernaderos. En cada uno observamos una cantidad de semillas y cuántas germinaron:

Invernadero	Semillas (N_i)	Germinadas (G_i)	Proporción observada
1	20	15	0.75
2	30	18	0.60
3	25	11	0.44
4	40	34	0.85

La idea es que cada invernadero puede tener su propia probabilidad de germinación, pero esas probabilidades no son completamente independientes: suponemos que los cuatro invernaderos pertenecen a una misma población.

a) Planteamiento del modelo jerárquico

Nivel 1: datos

Para cada invernadero $i = 1, \ldots, 4,$ tenemos N_i semillas y G_i germinaciones.

Por lo tanto:

$$G_i \sim Binomial(N_i, p_i)$$

donde:

- G_i: cantidad de semillas que germinaron;
- N_i: cantidad total de semillas evaluadas;
- p_i: probabilidad de germinación del invernadero i.

Por ejemplo, para el invernadero 1: $G_1 \sim Binomial(20, p_1)$

Nivel 2: probabilidades de los invernaderos

El problema es que p_i debe estar entre 0 y 1. Por eso trabajamos con la escala logística:

$$logit(p_i) = \alpha_i$$

o equivalentemente:

$$p_i = logit^{-1}(\alpha_i) = \frac{1}{1 + e^{-\alpha_i}}.$$

Ahora podemos modelar los efectos de cada invernadero:

$$\alpha_i \sim N(\mu, \sigma)$$

Esta es la parte jerárquica.

La interpretación es:

- α_i: efecto particular del invernadero i;
- μ: efecto promedio de la población de invernaderos;
- σ: cuánto varían los invernaderos alrededor de ese promedio.

En otras palabras, no suponemos que los cuatro invernaderos tienen exactamente la misma probabilidad de germinación, pero sí suponemos que sus efectos provienen de una distribución común.

Nivel 3: hiperparámetros

Necesitamos asignar priors a μ y σ.

Una elección razonable es:

$$\mu \sim N(0, 2)$$

y

$$\sigma \sim HalfNormal(2)$$

.

Entonces tenemos el siguiente resumen:

$$G_i \sim Binomial(N_i, p_i)$$

$$p_i \sim logit^{-1}(\alpha_i)$$

$$\alpha_i \sim N(\mu, \sigma)$$

$$\sigma \sim HalfNormal(2)$$

con $i = 1, \ldots, 4$.

¿Por qué este modelo es jerárquico?:

Porque tenemos distintos niveles:

Nivel de observación

$$G_i \mid p_i$$

Luego

$$\alpha_i \mid \mu, \sigma$$

y por último un "nivel poblacional"

$$\mu, \sigma$$

Observación: Los invernaderos comparten información a través de μ y σ, evitando que cada invernadero sea tratado como un problema completamente independiente.

b) **Construyamos le modelo:**

In [41]:
```
N = np.array([20, 30, 25, 40])
G = np.array([15, 18, 11, 34])
```

In [42]:
```
with pm.Model() as modelo:

    # Hiperparámetros
    mu = pm.Normal("mu", mu=0, sigma=2)
    sigma = pm.HalfNormal("sigma", sigma=2)

    # Efectos de cada invernadero
    alpha = pm.Normal(
        "alpha",
```

```
        mu=mu,
        sigma=sigma,
        shape=4
    )

    # Probabilidad de germinación
    p = pm.Deterministic(
        "p",
        pm.math.sigmoid(alpha)
    )

    # Datos observados
    germinadas = pm.Binomial(
        "germinadas",
        n=N,
        p=p,
        observed=G
    )
```

Generamos las muestras:

In [43]:
```
with modelo:
    idata = pm.sample(
        2000,
        tune=2000,
        chains=4,
        target_accept=0.95,
        random_seed=123
    )
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [mu, sigma, alpha]
```

Output()

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[43], line 2
      1 with modelo:
----> 2     idata = pm.sample(
      3         2000,
      4         tune=2000,
      5         chains=4,

File c:\Trabajos\Inferencia 2\venv\Lib\site-packages\pymc\sampling\mcmc.py:995, i
n sample(draws, tune, chains, cores, random_seed, progressbar, progressbar_theme,
quiet, step, var_names, nuts_sampler, initvals, init, jitter_max_retries, n_init,
trace, discard_tuned_samples, compute_convergence_checks, keep_warning_stat, retu
rn_inferencedata, idata_kwargs, nuts_sampler_kwargs, callback, mp_ctx, blas_core
s, model, compile_kwargs, **kwargs)
    991 t_sampling = time.time() - t_start
    993 # Packaging, validating and returning the result was extracted
    994 # into a function to make it easier to test and refactor.
--> 995 return _sample_return(
    996     run=run,
    997     traces=trace if isinstance(trace, ZarrTrace) else traces,
    998     tune=tune,
    999     t_sampling=t_sampling,
   1000     discard_tuned_samples=discard_tuned_samples,
   1001     compute_convergence_checks=compute_convergence_checks,
   1002     return_inferencedata=return_inferencedata,
   1003     keep_warning_stat=keep_warning_stat,
   1004     idata_kwargs=idata_kwargs or {},
   1005     model=model,
   1006     quiet=quiet,
   1007 )

File c:\Trabajos\Inferencia 2\venv\Lib\site-packages\pymc\sampling\mcmc.py:1123,
in _sample_return(run, traces, tune, t_sampling, discard_tuned_samples, compute_c
onvergence_checks, return_inferencedata, keep_warning_stat, idata_kwargs, model,
quiet)
   1121 # Pick and slice chains to keep the maximum number of samples
   1122 if discard_tuned_samples:
-> 1123     traces, length = _choose_chains(traces, tune)
   1124 else:
   1125     traces, length = _choose_chains(traces, 0)

File c:\Trabajos\Inferencia 2\venv\Lib\site-packages\pymc\backends\base.py:633, i
n _choose_chains(traces, tune)
    631 lengths = [max(0, len(trace) - tune) for trace in traces]
    632 if not sum(lengths):
--> 633     raise ValueError("Not enough samples to build a trace.")
    635 idxs = np.argsort(lengths)
    636 l_sort = np.array(lengths)[idxs]

ValueError: Not enough samples to build a trace.
```

¿Por qué `target_accept=0.95` ?

Porque en modelos jerárquicos puede ser más frecuente encontrar problemas de geometría posterior y divergencias.

Aumentar `target_accept` hace que NUTS utilice pasos más pequeños y puede ayudar a reducirlas.

Pero atención: no significa que poner `target_accept=0.95` haga que cualquier divergencia desaparezca mágicamente. Las divergencias son un diagnóstico que debemos revisar.

Hacemos un resumen de datos de la smuestras generadas:

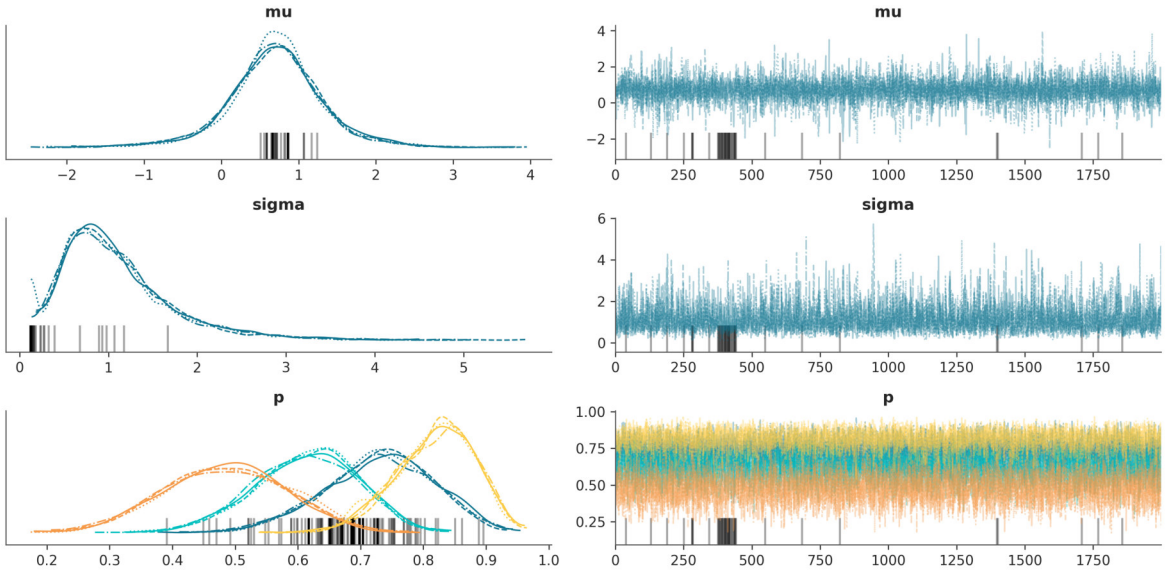
In [28]:
```
az.summary(
    idata,
    var_names=["mu", "sigma", "p", "alpha"],
    round_to=3
)
```

Out[28]:

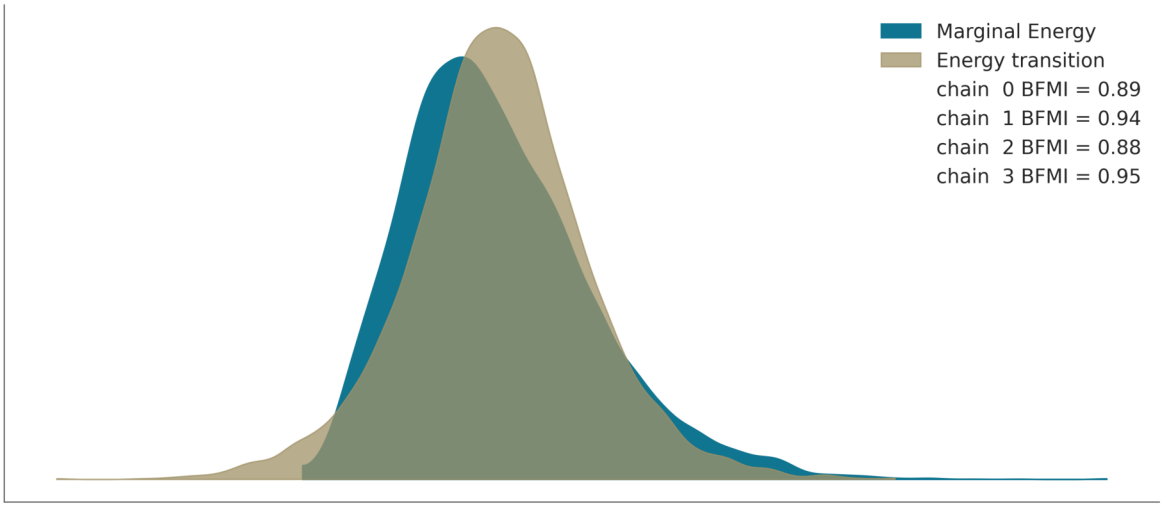
	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_h
mu	0.700	0.619	-0.501	1.943	0.010	0.011	4244.736	3720.349	1.0
sigma	1.098	0.628	0.124	2.215	0.012	0.011	1432.961	633.285	1.0
p[0]	0.729	0.086	0.568	0.888	0.001	0.001	6691.641	4932.363	1.0
p[1]	0.614	0.082	0.456	0.761	0.001	0.001	6752.266	5683.832	1.0
p[2]	0.486	0.098	0.314	0.674	0.001	0.001	4505.275	3440.891	1.0
p[3]	0.818	0.063	0.703	0.932	0.001	0.001	3086.276	822.472	1.0
alpha[0]	1.039	0.464	0.166	1.899	0.006	0.006	6691.641	4932.363	1.0
alpha[1]	0.476	0.356	-0.197	1.131	0.004	0.004	6752.266	5683.832	1.0
alpha[2]	-0.060	0.407	-0.781	0.729	0.006	0.004	4505.275	3440.891	1.0
alpha[3]	1.563	0.444	0.732	2.365	0.008	0.006	3086.276	822.472	1.0

In [29]:
```
az.plot_trace(
    idata,
    var_names=["mu", "sigma", "p"]
)
plt.tight_layout()
```

```
C:\Users\MAURICIO\AppData\Local\Temp\ipykernel_5104\2757621767.py:5: UserWarning:
The figure layout has changed to tight
  plt.tight_layout()
```



In [30]:
```
az.plot_energy(idata)
plt.show()
```



Y trabajamos con un breve análisis de divergencias:

In [31]:
```
divergencias = idata.sample_stats["diverging"].sum().item()

print("Número de divergencias:", divergencias)
```

```
Número de divergencias: 43
```

¿Qué hacemos si aparecen divergencias? Si bien en este ejemplo no vamos a tratar con demasiada profundidad, podemos hacer un esquema de cómo intentar reducir las divergencias del modelo.

Si obtenemos, por ejemplo:

`There were X divergences after tuning.`

no deberíamos simplemente ignorarlas.

Las divergencias indican que NUTS tuvo dificultades para explorar correctamente alguna región de la distribución posterior.

Podemos intentar cambiar `target_accept` :

In [32]:
```
#target_accept=0.95
```

In [33]:
```
#target_accept=0.99
```

por ejemplo:

In [34]:
```
with modelo:
    idata = pm.sample(
        2000,
        tune=3000,
        chains=4,
        target_accept=0.99,
        random_seed=123
    )
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [mu, sigma, alpha]
Output()
```

```
Sampling 4 chains for 3_000 tune and 2_000 draw iterations (12_000 + 8_000 draws
total) took 116 seconds.
There were 21 divergences after tuning. Increase `target_accept` or reparameteriz
e.
```

imprimimos las divergencias:

In [35]:
```
idata.sample_stats["diverging"].sum().item()
```

Out[35]: 21

Observen queInicialmente se observaron algunas divergencias durante el muestreo. Se aumentó target_accept para utilizar pasos de muestreo más conservadores. Luego de volver a ajustar el modelo, las divergencias disminuyeron, por lo que se continuará con la segunda muestra. Como no desaparecieron por completo, las estimaciones deben interpretarse con cautela dado que los posibles problemas de exploración de la distribución posterior y no deberían ignorarse como si fueran simplemente ruido computacional.

c) Resumen posterior

In [36]:
```
az.summary(
    idata,
    var_names=["p", "mu", "sigma"],
    hdi_prob=0.94,
    round_to=3
)
```

Out[36]:

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
p[0]	0.727	0.089	0.565	0.894	0.001	0.001	5232.199	4327.312	1.001
p[1]	0.613	0.082	0.450	0.755	0.001	0.001	5456.796	4807.892	1.001
p[2]	0.488	0.100	0.306	0.680	0.002	0.001	3229.451	1555.256	1.001
p[3]	0.817	0.062	0.700	0.927	0.001	0.001	3563.555	2270.936	1.001
mu	0.678	0.608	-0.487	1.857	0.010	0.011	4122.423	3830.311	1.001
sigma	1.094	0.626	0.119	2.241	0.012	0.012	1844.993	1167.842	1.003

Probabilidades p_i:

Tenemos:

$$p_1, p_2, p_3, p_4$$

que representan las probabilidades de germinación estimadas para cada invernadero.

No debemos confundirlas con las proporciones observadas.

Las proporciones observadas son:

$$(0.75, 0.60, 0.44, 0.85)$$

Las probabilidades posteriores p_i incorporan tanto los datos del propio invernadero como la información proveniente de los otros invernaderos.

μ:

representa el centro de la distribución de los efectos α_i.

Como μ está en escala logística, para interpretarlo en términos de probabilidad podemos transformar, después de extraer muestras:

In [37]:
```
mu_post = idata.posterior["mu"]

mu_prob = 1 / (1 + np.exp(-mu_post))
```

Así obtenemos una medida del nivel poblacional de germinación.

σ:

mide la variabilidad entre los invernaderos.

Una interpretación sencilla:

- σ pequeña → los invernaderos tienden a ser parecidos;
- σ grande → hay mayor heterogeneidad entre invernaderos.

Por eso σ es fundamental para entender cuánto pooling o contracción esperamos.

Podemos comparar las proporciones observadas con las estimaciones posteriores:

In [38]:
```
post = az.extract(idata)

p_mean = post["p"].mean("sample")

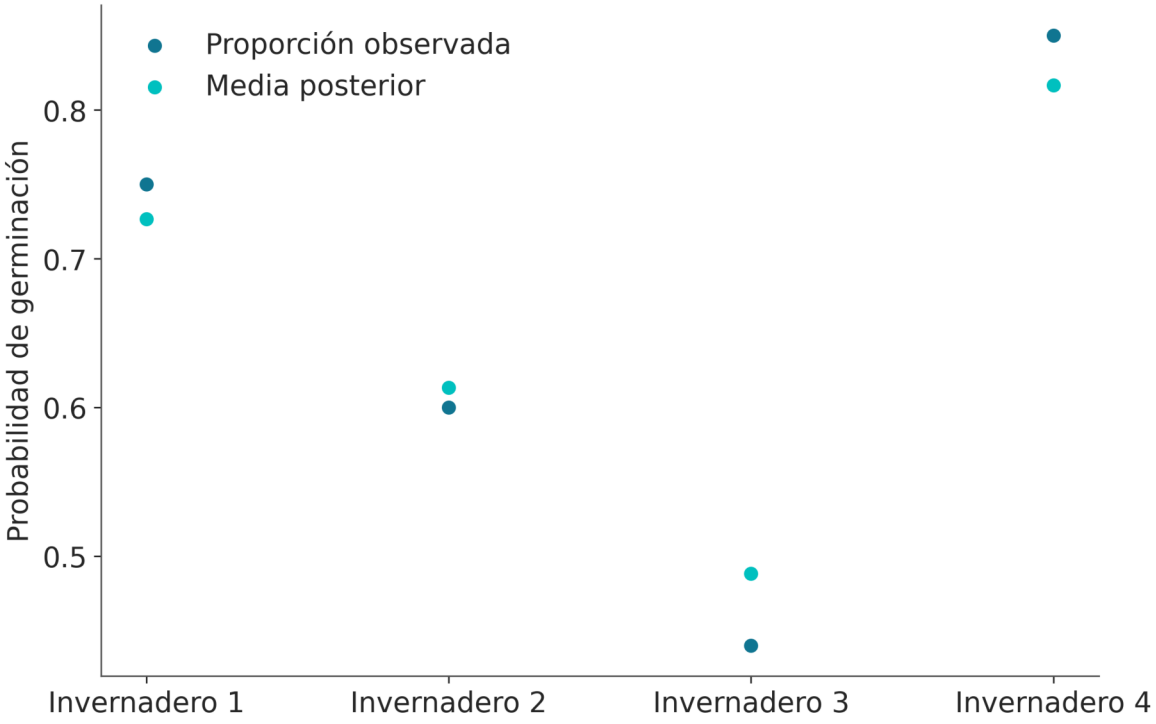
plt.figure(figsize=(8,5))

plt.scatter(
    np.arange(4),
    G / N,
    label="Proporción observada"
)

plt.scatter(
    np.arange(4),
    p_mean,
    label="Media posterior"
)

plt.xticks(
    np.arange(4),
    ["Invernadero 1", "Invernadero 2",
     "Invernadero 3", "Invernadero 4"]
)

plt.ylabel("Probabilidad de germinación")
plt.legend()
plt.show()
```



Aquí podemos empezar a visualizar el shrinkage.

d) **Comparación de los tres escenarios**

Ahora viene la parte más interesante.

Los primeros tres invernaderos permanecen iguales:

$$G = (15, 18, 11, \cdot)$$

y modificamos únicamente el cuarto.

Tenemos:

Escenario A

$$G_4 = 34$$

$$\frac{34}{40} = 0.85$$

Escenario B

$$G_4 = 34$$

$$\frac{8}{40} = 0.2$$

Escenario C

$$G_4 = 34$$

$$\frac{20}{40} = 0.50$$

Podemos crear una función para no repetir todo el código:

In [39]:
```
def ajustar_modelo(G, seed=123):

    G = np.array(G)

    with pm.Model() as modelo:

        mu = pm.Normal("mu", mu=0, sigma=2)
        sigma = pm.HalfNormal("sigma", sigma=2)

        alpha = pm.Normal(
            "alpha",
            mu=mu,
            sigma=sigma,
            shape=4
        )

        p = pm.Deterministic(
            "p",
            pm.math.sigmoid(alpha)
        )

        germinadas = pm.Binomial(
            "germinadas",
            n=N,
            p=p,
            observed=G
        )

        idata = pm.sample(
            2000,
            tune=2000,
            chains=4,
            target_accept=0.95,
            random_seed=seed
        )

    return idata
```

In [40]:
```
idata_A = ajustar_modelo([15, 18, 11, 34], seed=123)
idata_B = ajustar_modelo([15, 18, 11, 8], seed=456)
idata_C = ajustar_modelo([15, 18, 11, 20], seed=789)
```

```
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [mu, sigma, alpha]
```

Output()

```
Sampling 4 chains for 2_000 tune and 2_000 draw iterations (8_000 + 8_000 draws t
otal) took 59 seconds.
There were 43 divergences after tuning. Increase `target_accept` or reparameteriz
e.
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [mu, sigma, alpha]
```

Output()

```
Sampling 3 chains for 2_000 tune and 143 draw iterations (6_000 + 429 draws tota
l) took 33 seconds.
We recommend running at least 4 chains for robust computation of convergence diag
nostics
The rhat statistic is larger than 1.01 for some parameters. This indicates proble
ms during sampling. See https://arxiv.org/abs/1903.08008 for details
The effective sample size per chain is smaller than 100 for some parameters.  A h
igher number is needed for reliable rhat and ess computation. See https://arxiv.o
rg/abs/1903.08008 for details
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [mu, sigma, alpha]
```

Output()

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[40], line 3
      1 idata_A = ajustar_modelo([15, 18, 11, 34], seed=123)
      2 idata_B = ajustar_modelo([15, 18, 11, 8], seed=456)
----> 3 idata_C = ajustar_modelo([15, 18, 11, 20], seed=789)

Cell In[39], line 29, in ajustar_modelo(G, seed)
     25             p=p,
     26             observed=G
     27         )
     28
---> 29         idata = pm.sample(
     30             2000,
     31             tune=2000,
     32             chains=4,

File c:\Trabajos\Inferencia 2\venv\Lib\site-packages\pymc\sampling\mcmc.py:995, i
n sample(draws, tune, chains, cores, random_seed, progressbar, progressbar_theme,
quiet, step, var_names, nuts_sampler, initvals, init, jitter_max_retries, n_init,
trace, discard_tuned_samples, compute_convergence_checks, keep_warning_stat, retu
rn_inferencedata, idata_kwargs, nuts_sampler_kwargs, callback, mp_ctx, blas_core
s, model, compile_kwargs, **kwargs)
    991 t_sampling = time.time() - t_start
    993 # Packaging, validating and returning the result was extracted
    994 # into a function to make it easier to test and refactor.
--> 995 return _sample_return(
    996     run=run,
    997     traces=trace if isinstance(trace, ZarrTrace) else traces,
    998     tune=tune,
    999     t_sampling=t_sampling,
   1000     discard_tuned_samples=discard_tuned_samples,
   1001     compute_convergence_checks=compute_convergence_checks,
   1002     return_inferencedata=return_inferencedata,
   1003     keep_warning_stat=keep_warning_stat,
   1004     idata_kwargs=idata_kwargs or {},
   1005     model=model,
   1006     quiet=quiet,
   1007 )

File c:\Trabajos\Inferencia 2\venv\Lib\site-packages\pymc\sampling\mcmc.py:1123,
in _sample_return(run, traces, tune, t_sampling, discard_tuned_samples, compute_c
onvergence_checks, return_inferencedata, keep_warning_stat, idata_kwargs, model,
quiet)
   1121 # Pick and slice chains to keep the maximum number of samples
   1122 if discard_tuned_samples:
-> 1123     traces, length = _choose_chains(traces, tune)
   1124 else:
   1125     traces, length = _choose_chains(traces, 0)

File c:\Trabajos\Inferencia 2\venv\Lib\site-packages\pymc\backends\base.py:633, i
n _choose_chains(traces, tune)
    631 lengths = [max(0, len(trace) - tune) for trace in traces]
    632 if not sum(lengths):
--> 633     raise ValueError("Not enough samples to build a trace.")
    635 idxs = np.argsort(lengths)
    636 l_sort = np.array(lengths)[idxs]

ValueError: Not enough samples to build a trace.
```

Comparar el cuarto invernadero

Podemos obtener las medias posteriores:

In [ ]:
```
def media_p(idata):
    return (
        idata.posterior["p"]
        .mean(dim=("chain", "draw"))
        .values
    )

p_A = media_p(idata_A)
p_B = media_p(idata_B)
p_C = media_p(idata_C)

print("Escenario A:", p_A)
print("Escenario B:", p_B)
print("Escenario C:", p_C)
```

In [ ]:
```
print("Invernadero 4:")
print("A:", p_A[3])
print("B:", p_B[3])
print("C:", p_C[3])
```

¿Qué deberíamos observar?

No debemos esperar que la estimación posterior sea exactamente igual a la proporción observada.

En el escenario A:

$$\hat{p}_4^{obs} = 0.85$$

Pero el modelo probablemente produzca una estimación menor que 0.85, porque los otros tres invernaderos presentan:

$$0.75, 0.60, 0.44.$$

Es decir, el cuarto invernadero es relativamente alto respecto del resto.

El modelo lo "acerca" parcialmente hacia la población.

En el escenario B:

$$\hat{p}_4^{obs} = 0.2$$

Es una proporción bastante menor que la de los otros tres invernaderos.

El modelo jerárquico probablemente produzca una estimación mayor que 0.20.

Nuevamente aparece la contracción hacia el comportamiento general.

En el escenario C:

$$\hat{p}_4^{obs} = 0.5$$

Esta proporción ya está mucho más cerca del comportamiento de los otros invernaderos.

Por lo tanto, esperamos una menor corrección o contracción.

¿Por qué sucede esto?

Porque el modelo no piensa que el invernadero 4 es completamente independiente y tiene que estimar su probabilidad solamente con sus 40 semillas, sino que piensa que el invernadero 4 tiene sus propios datos, pero pertenece a una población de invernaderos que presenta cierto comportamiento general...

Entonces la estimación combina dos fuentes de información:

información propia + información de los demás invernaderos

Eso es precisamente el **pooling parcial**.

¿Cuál escenario tiene mayor contracción?

La contracción depende de varios factores:

1. Qué tan extremo es el grupo respecto del promedio poblacional.
2. Cuánta información tiene el grupo.
3. Cuánta variabilidad entre grupos permite el modelo, es decir, el valor de σ.

En este ejercicio, fíjense que el cuarto invernadero tiene $N_4 = 40$, que no es particularmente pequeño.

Por eso su información propia tiene bastante peso.

Aun así, entre los escenarios, A y B presentan observaciones más alejadas del centro de los otros invernaderos, por lo que veremos una modificación respecto de la proporción observada.

El escenario C está mucho más cerca del comportamiento general, por lo que la diferencia entre observación y posterior será menor.

Algunas observacioens sobre la contracción:

1. No es correcto decir simplemente que el grupo con la proporción más extrema siempre tiene mayor shrinkage.

2. El shrinkage no depende solamente de qué tan extrema sea la proporción.

3. También depende del tamaño muestral.

$$grupo\ A: \frac{2}{2} = 1.00$$

$$grupo\ B: \frac{40}{40} = 1.00$$

Ambos tienen la misma proporción observada, pero el segundo tiene mucha más evidencia. Por lo tanto, esperamos que el grupo con $2/2$ experimente mucho más shrinkage.

- menos información propia implica más influencia del promedio poblacional.
- más información propia implica menos contracción.

Intercambiabilidad

Cuando decimos que los invernaderos son intercambiables, no estamos diciendo que sean idénticos.

Estamos diciendo que, antes de observar los datos, no tenemos razones para pensar que un invernadero específico merece un tratamiento diferente de otro.

Por ejemplo, si tenemos: α_1 , α_2 , α_3 , α_4 , el modelo supone:

$$\alpha_i \sim N(\mu, \sigma)$$

No hay un invernadero que tenga un tratamiento especial.

Si permutáramos las etiquetas de los invernaderos, por ejemplo:

$(1, 2, 3, 4) \rightarrow (3, 1, 4, 2)$,

el modelo jerárquico sigue siendo esencialmente el mismo.

Observación: lo que importa es qué datos corresponden a cada invernadero, no que el número 1 tenga alguna característica especial por llamarse "invernadero 1".

¿Cuándo dejaría de ser razonable asumir intercambiabilidad?

Supongamos que sabemos que los invernaderos pertenecen a regiones climáticas completamente diferentes:

- algunos están en zonas frías;
- otros en zonas tropicales.
- que sabemos que:

1. algunos utilizan una tecnología de iluminación diferente;
2. algunos recibieron un tratamiento distinto;
3. algunos tienen un sistema de riego diferente;
4. algunos pertenecen a establecimientos diferentes;
5. hay una variable ambiental que sabemos que explica parte de las diferencias.

En esos casos, ya tenemos información que distingue a los grupos. Entonces no sería apropiado asumir que todos son intercambiables sin más.

Podríamos incorporar esas variables mediante un modelo jerárquico más elaborado.

Por ejemplo:

$$logit(p_i) = \alpha + \beta_1\, temperatura_i + \beta_2\, humedad_i$$

Y todavía podríamos mantener una estructura jerárquica si existen otros niveles.

Como conclusión: podemos considerar intercambiables a los cuatro invernaderos porque, antes de observar los resultados, suponemos que pertenecen a una misma población y no contamos con información que permita distinguir a uno de ellos de los demás. Por eso modelamos sus efectos mediante una distribución común. La intercambiabilidad no implica que tengan la misma probabilidad de germinación, sino que las diferencias entre ellos se consideran variaciones dentro de una misma población.

¿Qué ventajas tiene frente a estimar cada invernadero por separado?

Si hiciéramos cuatro modelos independientes:

$$G_i \sim Binomial(N_i, p_i)$$

y asignáramos a cada p_i un prior independiente, cada invernadero aprendería exclusivamente de sus propios datos.

Eso puede generar estimaciones muy extremas cuando tenemos pocos datos.

El modelo jerárquico permite:

1. Compartir información: porque los invernaderos informan indirectamente sobre los demás mediante μ y σ.
2. Reducir estimaciones extremas: un resultado como $0/2$ o $2/2$ no necesariamente produce una estimación posterior cercana a 0 o 1.
3. Mejorar la estimación de grupos pequeños: los grupos con poca información reciben más influencia de la población.
4. Mantener diferencias entre grupos: esto es importante porque no estamos haciendo pooling completo. No estamos diciendo:

$$p_1 = p_2 = p_3 = p_4$$

Cada invernadero conserva su propio p_i, y es por esto que hablamos de pooling parcial.